Der Daten-Lifecycle

Session 3: Mit Daten umgehen und zur Verfügung stellen

Simon Munzert
Hertie School

Überblick

1. Daten-Organisation in Spreadsheets
2. Relationale Datenstrukturen
3. Austauschformate: XML, JSON und die XÖV-Standards
4. Daten zur Verfügung stellen

Daten vs. Datenformate

Datenformate

- Datenformate sind die **Art und Weise, wie Daten gespeichert und übertragen werden**.
- Sie sind **nicht** dasselbe wie Datentypen.
- Datenformate legen fest, wie Daten strukturiert und abgelegt werden, und wie sie bei ihrer Verarbeitung zu interpretieren sind.
- Es gibt eine nahezu unerschöpfliche Masse an Datenformaten, die sich zum Beispiel in der Vielfalt von Dateinamenserweiterungen widerspiegelt.



Wahl von Datenformaten

- Datenwissenschaftler haben oft mehr oder weniger ausgeprägte Präferenzen für bestimmte Datenformate und -strukturen.
- **Beispiel 1** (für einfach strukturierte Daten): Excel Binary File Format (.XLS) < Office Open XML (.XLSX) < CSV)
- **Beispiel 2** (für semistrukturierte Daten): JSON vs. XML
- **Beispiel 3** (für vektorielle Geodaten): Shapefile vs. GeoJSON

Daten-Organisation in Spreadsheets

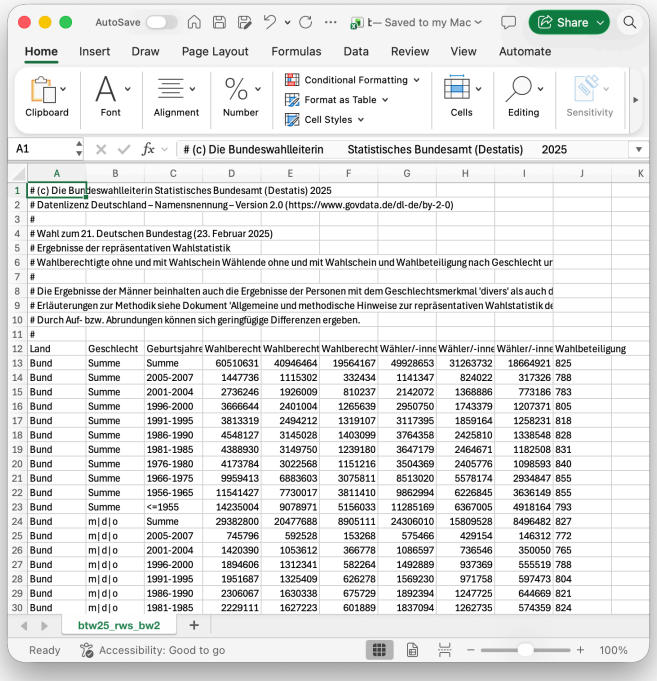
Die Verwaltung läuft auf Excel

Das verbreitetste Datenwerkzeug der Welt

- Excel ist in praktisch jeder Behörde installiert, jeder kann es öffnen und bedienen (..?). Deshalb ist das Spreadsheet die **Standardform der Datenhaltung** jenseits der Fachverfahren: Listen, Kennzahlen, Zuwendungen.
- Typischer Lebenszyklus einer Verwaltungstabelle: Export aus dem Fachverfahren, Bearbeitung in Excel, **Versand per E-Mail**, Rückmeldung in einer bearbeiteten Kopie, Zusammenführung von Hand, Weiterleitung nach oben.

Das Hin-und-her-Schubsen

- `Kennzahlen_2026_final.xlsx`, `Kennzahlen_2026_final_v2.xlsx`, `Kennzahlen_2026_final_v2_NEU_Korrektur_MS.xlsx`
- Welche Version stimmt? Wer hat was geändert? Wo ist das Original? Spalten werden umbenannt, Zeilen eingefügt, Formeln überschrieben, Formate „verschönert“. Jede Weitergabe ist ein **Medienbruch im Kleinen**.



`Quelle` Bundeswahlleiterin

Wenn das Spreadsheet die Datenbank ist

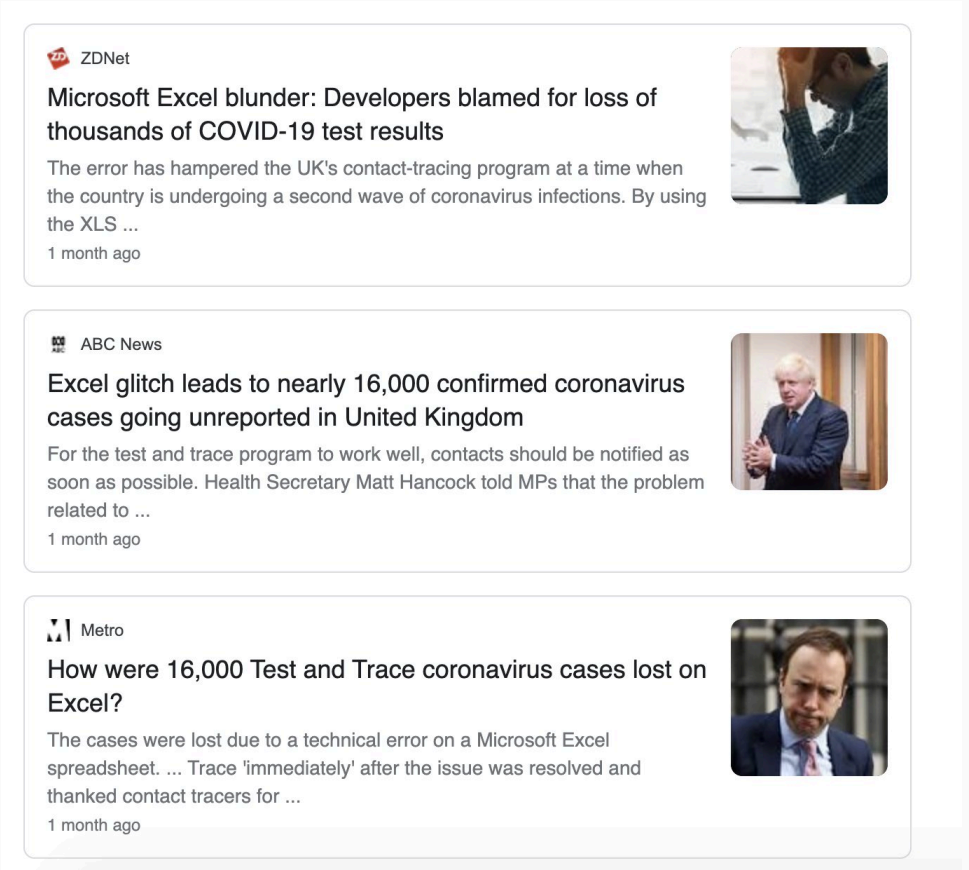
Hertie School

England, Oktober 2020

- Public Health England sammelte die Ergebnisse der Corona-Testlabore als CSV und überführte sie per Skript in **Excel-Vorlagen im alten XLS-Format**.
- XLS kann maximal **65.536 Zeilen**. Was darüber hinausging, wurde **stillschweigend abgeschnitten**.
- Zwischen dem 25. September und dem 2. Oktober 2020 fehlten so **15.841 positive Fälle** in der Meldung, rund 15 bis 20% aller Fälle. Ihre Kontakte wurden nicht nachverfolgt.

Die Folgen

- Fetzer und Graeber (2021) nutzen den Zufall des Fehlers als natürliches Experiment: In stärker betroffenen Regionen folgten mehr als **125.000 zusätzliche Infektionen** und über **1.500 zusätzliche Todesfälle**.



`Quellen` BBC (2020); The Conversation (2020)

Excel verändert Daten, ohne zu fragen

Automatische „Korrekturen“

- **Gen-Namen werden zu Datumsangaben**: `SEPT2` wird zu `2-Sep`, `MARCH1` zu `1-Mar`. Ziemann et al. (2016) fanden solche Fehler in rund **20% aller Genetik-Publikationen** mit Excel-Anhängen. 2020 wurden die Gene umbenannt, weil Excel nicht zu ändern war.
- **Führende Nullen verschwinden**: Die Postleitzahl `01067` (Dresden) wird zur Zahl `1067`. Das Gleiche passiert mit Aktenzeichen, Kontonummern, Gemeindeschlüsseln.
- **Lange Zahlen werden gerundet**: Ab 16 Stellen schneidet Excel ab; `1,23457E+15` statt der Kennnummer.
- **Trennzeichen**: Deutsche Excel-Versionen lesen CSV mit Komma als eine einzige Spalte, weil das Komma hier Dezimaltrenner ist.

Was daraus folgt

- Excel **interpretiert** Daten beim Öffnen. Was Ihr seht, ist nicht unbedingt, was gespeichert ist.
- Jede Übergabe einer Tabelle zwischen Rechnern, Sprachversionen und Programmen ist eine **potenzielle Fehlerquelle**.
- Wer Daten **weitergibt**, sollte sie in einem Format weitergeben, das nicht interpretiert wird (Text, siehe gleich), und dokumentieren, was die Spalten bedeuten.



Gute Tabellen: zwölf Regeln (1/2)

1. **Seid konsistent.** Gleiche Schreibweise für gleiche Dinge: nicht `m`, `männlich`, `M`, `Mann` in einer Spalte. Gleiche Kennung für fehlende Werte (leer oder `NA`), gleiche Dateinamen-Logik, gleiche Spaltennamen in allen Tabellen.

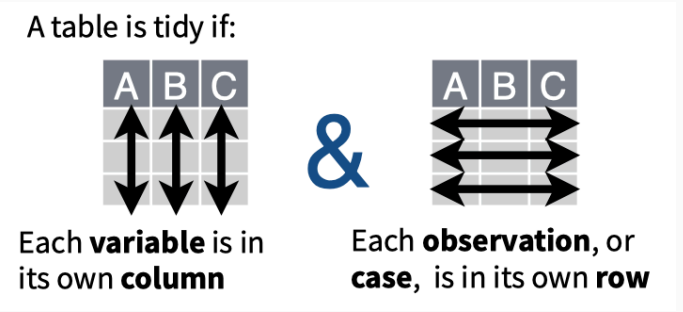
2. **Schreibt Datumsangaben als `JJJJ-MM-TT`.** `2026-10-27` ist eindeutig, sortierbar und international (ISO 8601). Alles andere wird früher oder später falsch gelesen.

3. **Lasst keine Zellen leer.** Leer heißt: unbekannt, nicht zutreffend, vergessen oder „siehe oben“? Schreibt es hin.

4. **Eine Information pro Zelle.** Nicht `Berlin, 10117` in einer Zelle, sondern zwei Spalten. Keine Einheiten in der Zahl (`12 km`), sondern eine Spalte `distanz_km`.

5. **Macht ein Rechteck daraus.** Eine Kopfzeile, darunter eine Zeile pro Beobachtung, eine Spalte pro Merkmal. Keine verbundenen Zellen, keine Zwischenüberschriften, keine Summenzeilen im Datenbereich, keine zwei Tabellen auf einem Blatt.

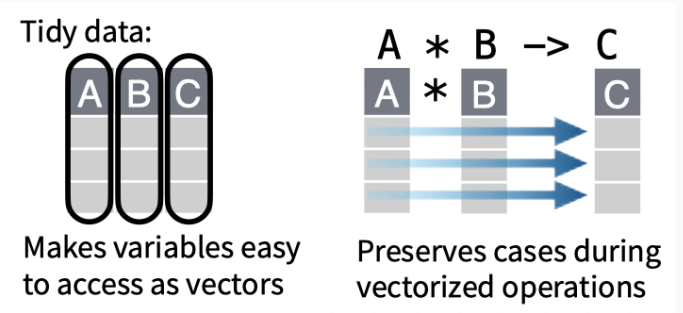
6. **Erstellt ein Datenwörterbuch.** Eine zweite Tabelle: Spaltenname, Bedeutung, Einheit, erlaubte Werte, Quelle.



`Quelle` Broman und Woo (2018), Data organization in spreadsheets, The American Statistician, frei lesbar unter kbroman.org/dataorg.

Gute Tabellen: zwölf Regeln (2/2)

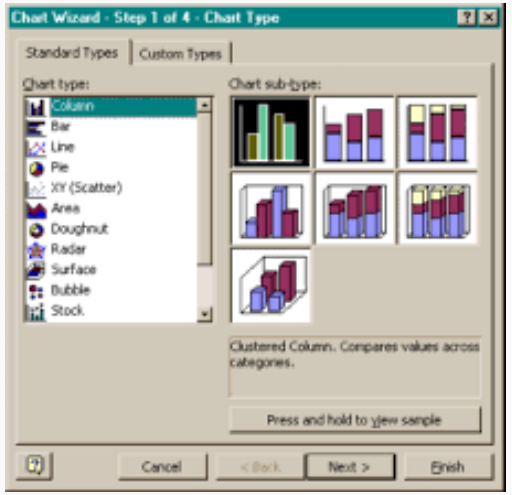
7. **Keine Berechnungen in den Rohdaten.** Rohdaten sind unantastbar. Summen, Prozente, Diagramme kommen in eine separate Datei oder ein separates Blatt.
8. **Keine Farben und Formatierungen als Information.** „Rot markiert heißt geprüft“ ist für Menschen lesbar, für Maschinen unsichtbar und beim nächsten Export weg. Macht eine Spalte `geprueft` mit `ja` / `nein`.
9. **Wählt gute Namen.** Spaltennamen ohne Leerzeichen, Umlaute und Sonderzeichen: `antrag_datum` statt `Datum des Antrags (neu)`. Kurz, aber sprechend.
10. **Macht Sicherungskopien.** Und schützt die Rohdaten vor Überschreiben (schreibgeschützt ablegen).
11. **Nutzt Datenvalidierung bei der Eingabe.** Excel kann Dropdown-Listen und Wertebereiche erzwingen. Das verhindert `Bayenr` und `31.02.2026`.
12. **Speichert Daten als Text.** CSV (oder TSV) ist das einzige Format, das in 20 Jahren noch jedes Programm liest. Excel-Dateien sind Arbeitsdateien, keine Archivdateien.



✎ Die meisten dieser Regeln laufen auf eines hinaus: **Trennt die Daten von ihrer Darstellung.**

Woran man erkennt, dass eine Tabelle zur Datenbank geworden ist

- Mehrere Personen bearbeiten sie, und es gibt Konflikte.
- Sie wird regelmäßig aktualisiert und an andere verschickt.
- Andere Tabellen oder Berichte hängen davon ab.
- Es gibt mehr als eine Version, und niemand weiß, welche gilt.
- Sie hat mehr als ein paar tausend Zeilen.



Stufen der Besserung

1. **Eine Quelle der Wahrheit**: Eine Datei an einem Ort, nicht per Mail; Änderungen nur dort.
2. **Trennung von Erfassung und Auswertung**: Rohdaten-Tabelle plus separate Auswertungsdateien.
3. **Geteilte Tabelle mit Rechten** (SharePoint, Nextcloud, Google Sheets): gleichzeitiges Arbeiten, Änderungshistorie.
4. **Datenbank mit Eingabemaske**: Wenn Regeln, Mehrbenutzerzugriff und Verknüpfungen nötig sind.

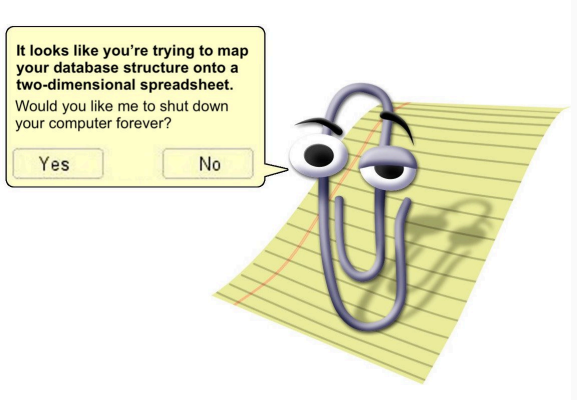
✎ Die Frage ist nicht „Excel oder nicht Excel“, sondern: **Wer ist für diese Daten verantwortlich, und wo ist das Original?**

Relationale Datenstrukturen

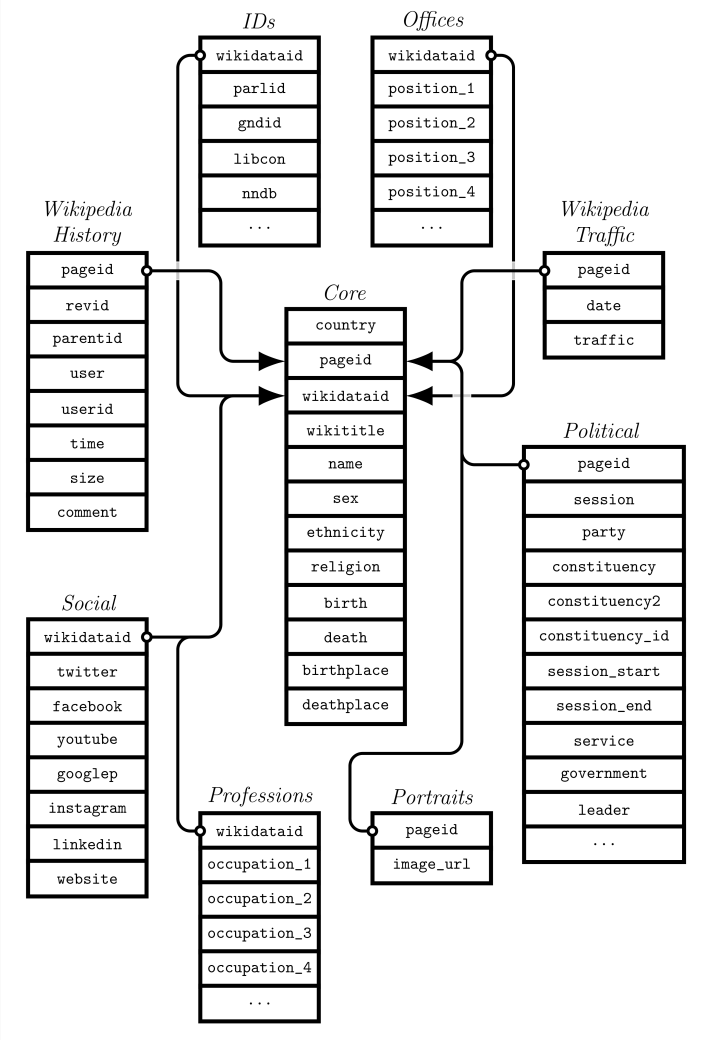
Die Allgegenwärtigkeit von relationalen Datenstrukturen

Von Tabellen...

- Wenn Ihr an Daten auf Eurem Laptop denkt, habt Ihr möglicherweise ein Spreadsheet im Kopf.
- In der Tat arbeitet ein Großteil der klassischen "statistischen" Software (SPSS, Stata, MS Excel) standardmäßig mit Spreadsheets als Dateninput.
- Gleichzeitig ist einer Eurer ersten Gedanken möglicherweise auch dateibasiert. Die Daten werden in Dateien gespeichert, und diese Dateien werden von unserer Datenverwaltungssoftware gelesen (und erstellt).
- In vielen Fällen ist eine **zweidimensionale Datenstruktur** sinnvoll. Wir nutzen zum Beispiel
  - Befragte x Einstellungen
  - Bezirke x Merkmale
  - Beiträge in sozialen Medien x Textmerkmale



Die Allgegenwärtigkeit von mehrdimensionalen Datenstrukturen



Credit saschagobel/legislatoR

... zu komplexen Datenstrukturen

- Je länger man jedoch darüber nachdenkt, desto problematischer wird es, Daten systematisch in zweidimensionalen Strukturen abzulegen.
- **Beispiele**:
  - Befragte x Einstellungen x Länder x Zeit
  - Bezirke x Unterbezirke Merkmale x Zeit
  - Beiträge in sozialen Medien x Retweets x Benutzer x Benutzereigenschaften x Netzwerkmerkmale x Metadaten
- Die Abbildung von dreidimensionalen auf zweidimensionale Strukturen ist technisch einfach machbar.
- Bei **vielfachen heterogenen Datenquellen** wird es unübersichtlich.
- Die Verwaltung komplexer Datenstrukturen ist nur ein Vorteil der Verwendung von **Datenbanken**.

Datenbanken

Was sind Datenbanken?

- Datenbanken sind eine organisierte Sammlung von Daten.
- Sie sind so organisiert, dass sie ein effizientes Abrufen von Daten(-selektionen) ermöglichen.
- Sie enthalten Daten + Metadaten über Struktur und Organisation.
- Der Zugriff auf sie erfolgt im Allgemeinen über ein Datenbankmanagementsystem.

Wo sind Datenbanken?

- Datenbanken können lokal oder dezentral, im Arbeitsspeicher oder auf der Festplatte existieren.
- Wenn sie lokal gespeichert sind, werden sie als Binärdatei (nicht als Textdatei) gespeichert.
- Im Allgemeinen denken wir an ein **Client-Server-Modell**:
  - Datenbanken befinden sich auf einem **Server**, der sie verwaltet.
  - Die Benutzer interagieren mit dem Server über ein **Client**-Programm.
  - Mehrere Benutzer können auf dieselbe Datenbank **gleichzeitig zugreifen**.

Wann Datenbanken nützlich werden

Größe und Struktur

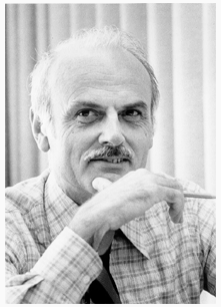
- Ihr habt **Datenmengen, die den Arbeitsspeicher** auf Eurem Computer übersteigen. Datenbanken sind nur durch die verfügbare Festplattengröße begrenzt (oder können auf mehrere Festplatten/Rechner verteilt werden) und lassen sich schnell und häufig abfragen.
- Eure **Datenstruktur ist komplex**. Datenbanken erlauben es, Daten mit komplexen Datenstrukturen zu speichern, abzurufen und zu untergliedern.
- Ihr legt Wert auf **Datenqualität** und habt klare Vorstellungen, wie Daten aussehen sollen. Mit Hilfe von Datenbanken könnt Ihr spezifische Regeln für die Erweiterung und Aktualisierung Eurer Datenbank definieren.

Zugänglichkeit und Gleichzeitigkeit

- Ihr **arbeitet gemeinsam mit anderen** an einem Datenprojekt. Mit einer Datenbank habt Ihr eine gemeinsame, zuverlässige Infrastruktur zur Verfügung, auf die mehrere Benutzer gleichzeitig zugreifen können.
- Wenn mehrere Parteien beteiligt sind, können Nutzerprivilegien wichtig sein (z. B. nur Lesezugriff, Zugriff auf Teile der Daten, eingeschränkte Verwaltungsrechte usw.). Die meisten Datenbanken erlauben die **Definition unterschiedlicher Nutzungsrechte für verschiedene Benutzer**.

Codds relationales Modell für Datenbanken

- Das Konzept relationaler Datenbanken baut auf dem relational model (RM) auf, das 1969/1970 von Edgar F. "Ted" Codd vorgeschlagen wurde.
- Codd beschrieb das RM formal, führte es aber auch mit Konzepten ein, die auch heute noch verwendet werden (Normalisierung, Schlüssel, Joins, usw.).
- Die Grundannahme des relationalen Modells ist, dass alle Daten als Relationen (Tabellen) dargestellt werden können.
- Informationen werden dann durch in Beziehung stehende Datenwerte dargestellt.



A Relational Model of Data for Large Shared Data Banks

E. F. Codd
IBM Research Laboratory, San Jose, California

Future users of large data banks must be protected from having to know how the data is organized in the machine (the internal representation). A prompting service which supplies such information is not a satisfactory solution. Activities of users at terminals and most application programs should remain unaffected when the internal representation of data is changed and even when some aspects of the external representation are changed. Changes in data representation will often be needed as a result of changes in query, update, and report traffic and natural growth in the types of stored information.

Existing noninferential, formatted data systems provide users with tree-structured files or slightly more general network models of the data. In Section 1, inadequacies of these models are discussed. A model based on *n*-ary relations, a normal form for data base relations, and the concept of a universal data sublanguage are introduced. In Section 2, certain operations on relations (other than logical inference) are discussed and applied to the problems of redundancy and consistency in the user's model.

KEY WORDS AND PHRASES: data bank, data base, data structure, data organization, hierarchies of data, networks of data, relations, derivability, redundancy, consistency, composition, join, retrieval language, predicate calculus, security, data integrity
CR CATEGORIES: 3.70, 3.73, 3.75, 4.20, 4.22, 4.29

1. Relational Model and Normal Form

1.1. Introduction

This paper is concerned with the application of elementary relation theory to systems which provide shared access to large banks of formatted data. Except for a paper by Childs [1], the principal application of relations to data systems has been to deductive question-answering systems. Levein and Maron [2] provide numerous references to work in this area.

The relational view (or model) of data described in Section 1 appears to be superior in several respects to the graph or network model [3, 4] presently in vogue for noninferential systems. It provides a means of describing data with its natural structure only—that is, without superimposing any additional structure for machine representation purposes. Accordingly, it provides a basis for a high level data language which will yield maximal independence between programs on the one hand and machine representation and organization of data on the other.

A further advantage of the relational view is that it forms a sound basis for treating derivability, redundancy, and consistency of relations—these are discussed in Section 2. The network model, on the other hand, has spawned a number of confusions, not the least of which is mistaking the derivation of connections for the derivation of relations (see remarks in Section 2 on the "connection trap").

Finally, the relational view permits a clearer evaluation of the scope and logical limitations of present formatted data systems, and also the relative merits (from a logical standpoint) of competing representations of data within a single system. Examples of this clearer perspective are cited in various parts of this paper. Implementations of systems to support the relational model are not discussed.

1.2. Data Dependencies in Present Systems

The provision of data description tables in recently developed information systems represents a major advance toward the goal of data independence [5, 6, 7]. Such tables facilitate changing certain characteristics of the data representation stored in a data bank. However, the variety of data representation characteristics which can be changed *without logically impairing some application programs* is still quite limited. Further, the model of data with which users interact is still cluttered with representational properties, particularly in regard to the representation of collections of data (as opposed to individual items). Three of the principal kinds of data dependencies which still need to be removed are: ordering dependence, indexing dependence, and access path dependence. In some systems these dependencies are not clearly separable from one another.

1.2.1. *Ordering Dependence.* Elements of data in a data bank may be stored in a variety of ways, some involving no concern for ordering, some permitting each element to participate in one ordering only, others permitting each element to participate in several orderings. Let us consider those existing systems which either require or permit data elements to be stored in at least one total ordering which is

`Credit` Communications of the ACM 13(6), 1970

Tabellarische Daten: Beispiel

Speicherung von Daten in Tabellen

- Eine einzige Tabelle stellt bereits Beziehungen her: Alle Daten in einer Zeile gehören zu demselben Datensatz.
- Wenn wir komplexere Beziehungen darstellen wollen (z. B. das Gewicht einer Person zweimal messen oder das Gewicht ihrer Kinder ebenfalls messen), können wir Daten von einer Tabelle zur anderen in Beziehung setzen.

Beispiel

- Wir haben Daten über Peter, Paul und Mary gesammelt.
- Wir haben Informationen über Geburtstage, Telefonnummern und Lieblingsspeisen.
- Wie können wir diese Informationen in Tabellen darstellen?

Table 7.1 Our friends' birthdays and favorite foods

nameid	name	birthday	favoritefood1	favoritefood2	favoritefood3
1	Peter Pascal	01/02/1991	spaghetti	hamburger	
2	Paul Panini	02/03/1992	fruit salad		
3	Mary Meyer	03/04/1993	chocolate	fish fingers	hamburger

Table 7.2 Our friends' telephone numbers

telephoneid	nameid	telephonenumber
1	1	001665443
2	2	00255555
3	1	001878345

- Wir beginnen mit der Darstellung der Daten in zwei Tabellen.
- Sie sind über den Schlüssel `nameid` verknüpft, so dass wir die vollständigen Namen nicht in die Tabelle der Telefonnummern aufnehmen müssen.
- Beachtet, dass wir hier eine 1:m (one-to-many) Beziehung haben, da Peter zwei Telefonnummern hat.

Tabellarische Daten: Beispiel

Speicherung von Daten in Tabellen

- Eine einzige Tabelle stellt bereits Beziehungen her: Alle Daten in einer Zeile gehören zu demselben Datensatz.
- Wenn wir komplexere Beziehungen darstellen wollen (z. B. das Gewicht einer Person zweimal messen oder das Gewicht ihrer Kinder ebenfalls messen), können wir Daten von einer Tabelle zur anderen in Beziehung setzen.

Beispiel

- Wir haben Daten über Peter, Paul und Mary gesammelt.
- Wir haben Informationen über Geburtstage, Telefonnummern und Lieblingsspeisen.
- Wie können wir diese Informationen in Tabellen darstellen?

Table 7.1 Our friends' birthdays and favorite foods

nameid	name	birthday	favoritefood1	favoritefood2	favoritefood3
1	Peter Pascal	01/02/1991	spaghetti	hamburger	
2	Paul Panini	02/03/1992	fruit salad		
3	Mary Meyer	03/04/1993	chocolate	fish fingers	hamburger

Table 7.2 Our friends' telephone numbers

telephoneid	nameid	telephonenumber
1	1	001665443
2	2	00255555
3	1	001878345

- Die Art und Weise, wie wir die Daten speichern, ist jedoch nicht ideal. In der ersten Tabelle haben wir drei Spalten, die praktisch das Gleiche messen. Und was, wenn es noch mehr Lieblingsessen gibt? Das Hinzufügen von Informationen auf diese Weise führt zu einer Menge redundanter Informationen.
- Wir können die Daten normalisieren, um Redundanzen zu vermeiden.

Tabellarische Daten: Beispiel

Speicherung von Daten in Tabellen

- Eine einzige Tabelle stellt bereits Beziehungen her: Alle Daten in einer Zeile gehören zu demselben Datensatz.
- Wenn wir komplexere Beziehungen darstellen wollen (z. B. das Gewicht einer Person zweimal messen oder das Gewicht ihrer Kinder ebenfalls messen), können wir Daten von einer Tabelle zur anderen in Beziehung setzen.

Beispiel

- Wir haben Daten über Peter, Paul und Mary gesammelt.
- Wir haben Informationen über Geburtstage, Telefonnummern und Lieblingsspeisen.
- Wie können wir diese Informationen in Tabellen darstellen?

Table 7.3 Our friends' birthdays—revised

name id	first name	last name	birthday
1	Peter	Pascal	01/02/1991
2	Paul	Panini	02/03/1992
3	Mary	Meyer	03/04/1993

Table 7.4 Our friends' food preferences

nameid	foodname	rank
1	spaghetti	1
1	hamburger	2
2	fruit salad	1
3	chocolate	1
3	fish fingers	2
3	hamburger	3

- Besser ist es, die Informationen aufzuteilen und eine weitere Tabelle für die Essenspräferenzen zu erstellen.
- Es bleibt immer noch etwas Redundanz übrig. Ist es wirklich notwendig, "Hamburger" zweimal in der Tabelle zu haben?

Tabellarische Daten: Beispiel

Speicherung von Daten in Tabellen

- Eine einzige Tabelle stellt bereits Beziehungen her: Alle Daten in einer Zeile gehören zu demselben Datensatz.
- Wenn wir komplexere Beziehungen darstellen wollen (z. B. das Gewicht einer Person zweimal messen oder das Gewicht ihrer Kinder ebenfalls messen), können wir Daten von einer Tabelle zur anderen in Beziehung setzen.

Beispiel

- Wir haben Daten über Peter, Paul und Mary gesammelt.
- Wir haben Informationen über Geburtstage, Telefonnummern und Lieblingsspeisen.
- Wie können wir diese Informationen in Tabellen darstellen?

Table 7.5 Our friend's food preferences—revised

rankid	nameid	foodid	rank
1	1	1	1
2	1	2	2
3	2	3	1
4	3	4	1
5	3	5	2
6	3	2	3

Table 7.6 Food types

foodid	food name	healthy	kcalp100g
1	spaghetti	no	0.158
2	hamburger	no	0.295
3	fruit salad	yes	0.043
4	chocolate	no	0.546
5	fish fingers	no	0.290

- Jetzt ist es besser.
- Bei der Umstrukturierung der Informationen in unserer Datenbank haben wir **Redundanz (Duplikation)** vermieden.
- Dies ist der Prozess der **Datenbanknormalisierung**.

Was sind DBMS?

- Datenbankmanagementsysteme (DBMS) bieten **effiziente**, **zuverlässige**, **bequeme**, **sichere** Speicherung von und Zugriff auf **massive** Daten.
  - **Effizient**: Einfach schnell.
  - **Zuverlässig**: Hohe Betriebszeit.
  - **Komfortabel**: Hochwertige Abfragesprachen.
  - **Sicher**: Robust gegenüber Stromausfällen, Knotenausfällen, usw.
  - **Mehrbenutzer**: Gleichzeitigkeitskontrolle. Nicht nur ein Benutzer, sondern mehrere.
  - **Massiv**: Denkt an Terabytes, nicht an Gigabytes. Verarbeitung von Daten, die sich außerhalb des Speichers befinden.
- Es gibt etliche DBMS allein für relationale Datenbankstrukturen.
- RDBMS unterscheiden sich in Bezug auf Fähigkeiten, implementierte Funktionen, Betriebssystemunterstützung und vieles mehr. Siehe The Database Database für einen Überblick.









Datenbanken in der Verwaltung: Wo liegen die Daten?

Fachverfahren statt Datenbank

- Behördendaten stecken fast immer in **Fachverfahren**: Software für einen Aufgabenbereich (Meldewesen, Kfz-Zulassung, Sozialleistungen, Bauanträge). Darunter läuft ein DBMS, das der Fachbereich nie sieht: Er bedient Masken, nicht Tabellen.
- Betrieb meist beim **IT-Dienstleister** (Dataport, AKDB, regio iT, ekom21 …) oder beim Hersteller. Wer die Daten auswerten will, braucht also einen Zugang, der im Vertrag steht, nicht nur eine Exportfunktion.

Register: viele, dezentral, nicht vernetzt

- Deutschland hat **über 375 Registertypen**, historisch gewachsen und föderal verteilt: das Melderegister bei jeder Gemeinde, das Handelsregister bei den Amtsgerichten, das Fahrzeugregister beim KBA (ÖFIT 2025, IT-Planungsrat 2021).
- Die Trennung ist **gewollt**: Das Volkszählungsurteil (1983) verbietet ein einheitliches Personenkennzeichen und verlangt Zweckbindung. Jede Verknüpfung über Register hinweg braucht eine Rechtsgrundlage.

Herstellerabhängigkeit

- Der Bund hat 2023 einen **Rahmenvertrag über 4,6 Mrd. €** mit Oracle geschlossen (Laufzeit bis 2030); laut einer Deloitte-Analyse für das BMI laufen rund 75% der Daten der Bundesverwaltung über Oracle-Produkte (Computerwoche 2023, Tagesspiegel Background 2024). Open-Source-DBMS wie PostgreSQL gelten als Weg zu mehr digitaler Souveränität.

Registermodernisierung: vernetzen statt zusammenlegen

Hertie School

Ziel: Once-Only

- Bürger und Unternehmen sollen Nachweise **nur einmal** vorlegen; die Verwaltung holt sie sich bei der zuständigen Stelle. Grundlage: EU-Single-Digital-Gateway-Verordnung (2018) und Registermodernisierungsgesetz (2021).
- Die **Steuer-ID wird Identifikationsnummer** für 51 priorisierte Register (Melde-, Personenstands-, Fahrzeugregister …), Frist Ende 2028.

Architektur: NOOTS

- Keine zentrale Datenbank, sondern ein **Vermittlungssystem** (Nationales Once-Only-Technical-System): Register bleiben, wo sie sind; Nachweise werden einzeln, mit Einwilligung und protokolliert abgerufen.
- Das **Datenschutzcockpit** zeigt Bürgern, welche Behörde wann welche Daten abgerufen hat.

Stand

- **14.01.2026**: erster Nachweisabruf über das NOOTS (Fahrzeugregister des KBA für einen Bewohnerparkausweis in Baden-Württemberg); Gewerbeanmeldung mit Handelsregister-Abruf in NRW folgt (BVA).
- Flächen-Rollout ab **November 2026**; Betrieb beim Bundesverwaltungsamt.

Kritik

- Die Steuer-ID als **Quasi-Personenkennzeichen** widerspreche dem Volkszählungsurteil; die Humanistische Union klagt (HU). Österreich nutzt bereichsspezifische Kennungen.

Vier Fragen, die Ihr Eurer IT oder Eurem Dienstleister auch ohne technisches Detailwissen stellen könnt

1. **Wo liegen unsere Daten?**
   Welche Fachverfahren nutzen wir, welche Daten stecken darin, und wer betreibt sie: unser eigenes Rechenzentrum, ein kommunaler IT-Dienstleister, der Hersteller oder ein Cloud-Anbieter?
2. **Kommen wir an unsere Daten heran?**
   Können wir sie vollständig und in einem offenen Format (z. B. CSV, XML) herausholen, auch ohne den Hersteller zu fragen? Gibt es eine dokumentierte Schnittstelle? Und steht das so im Vertrag? Wer das erst beim Anbieterwechsel klärt, zahlt doppelt.
3. **Können wir die Daten auswerten, ohne den laufenden Betrieb zu stören?**
   Das System, in dem die Sachbearbeitung täglich arbeitet, ist nicht zum Auswerten da. Gibt es eine Kopie der Daten für Analysen und Berichte, und wer darf darauf zugreifen?
4. **Worauf passt die Datenbank selbst auf?**
   Welche Fehler verhindert sie automatisch (Pflichtfelder, feste Auswahllisten, keine doppelten Einträge), und welche werden nur in Schulungen angesprochen? Wer darf was sehen und ändern, und wird das protokolliert?

✎ Datenbank-Kompetenz auf Leitungsebene heißt nicht, SQL-Abfragen zu schreiben, sondern zu wissen, **wo die Daten liegen, wem sie gehören und wer sie herausbekommt**. Alles andere lässt sich beauftragen.

Austauschformate: XML, JSON und die XÖV-Standards

Warum Austauschformate?

Excel ist kein Austauschformat

- Eine Excel-Datei transportiert **Darstellung** (Formate, Farben, Spaltenbreiten, Formeln).
- Für den Austausch zwischen **Programmen** braucht man Formate, die eindeutig Daten und deren Struktur transportieren.

Was ein gutes Austauschformat kann

- **Maschinenlesbar** ohne Rätselraten: Struktur ist explizit.
- **Validierbar**: Man kann prüfen, ob eine Datei den Regeln entspricht (Schema).
- **Offen dokumentiert**: Keine Abhängigkeit von einem Hersteller.
- **Textbasiert**: In jedem Editor lesbar, versionierbar, langlebig.

Drei Familien

- **Tabellen als Text**: CSV, TSV. Eine Zeile pro Datensatz, Trennzeichen zwischen Feldern. Einfach, universell, aber nur für flache Tabellen.
- **Hierarchische Formate**: XML, JSON. Können verschachtelte Strukturen abbilden (z.B. ein Antrag mit mehreren Anlagen). Das Format der Web-APIs und der Verwaltungsschnittstellen.
- **Binäre Formate**: Parquet, Datenbankdateien. Effizient für große Datenmengen, aber nur mit Spezialsoftware lesbar.



XML: Extensible Markup Language

Grundidee

- Daten werden in **Elemente** eingeschlossen, die mit Tags markiert sind: `<Name>Mustermann</Name>`.
- Elemente können **verschachtelt** werden und **Attribute** tragen.
- Ein **Schema** (XSD) legt fest, welche Elemente erlaubt sind, in welcher Reihenfolge, mit welchen Datentypen. Das lässt automatische **Validierung** zu.
- **Namensräume** (`xmlns`) verhindern, dass `<Name>` in zwei Standards etwas Verschiedenes bedeutet.

Wo XML lebt

- Fast alle **XÖV-Standards** der deutschen Verwaltung, die elektronische Rechnung (XRechnung), Office-Dokumente (DOCX ist gezipptes XML), RSS-Feeds, viele ältere Web-Schnittstellen (SOAP).

Beispiel

```xml
<?xml version="1.0" encoding="UTF-8"?>
<Personen>
  <Person id="1">
    <Name>van Pelt, Lucy</Name>
    <Geschlecht>weiblich</Geschlecht>
    <Alter>32</Alter>
  </Person>
  <Person id="2">
    <Name>Peppermint, Patty</Name>
    <Geschlecht>weiblich</Geschlecht>
    <Alter/>
  </Person>
  <Person id="3">
    <Name>Brown, Charlie</Name>
    <Geschlecht>männlich</Geschlecht>
    <Alter>27</Alter>
  </Person>
</Personen>
```

JSON: JavaScript Object Notation

Grundidee

- Daten stehen in **Schlüssel-Wert-Paaren**: `"name": "Brown, Charlie"`. Schlüssel immer in Anführungszeichen, Werte nur, wenn sie Text sind.
- Geschweifte Klammern `{ }` fassen ein **Objekt** zusammen (ein Datensatz), eckige Klammern `[ ]` eine **Liste** (mehrere Datensätze). Beides lässt sich beliebig **verschachteln**.
- Kein Schema vorgeschrieben, aber möglich (JSON Schema). Kompakter als XML, für Menschen gut lesbar.

Wo JSON lebt

- Praktisch jede **Web-API** und jedes **Open-Data-Portal** antwortet in JSON (Session 4). Konfigurationsdateien, moderne Schnittstellen zwischen Programmen, die Metadaten bei FIT-Connect.
- Einlesen ist in jeder Sprache ein Einzeiler, in R etwa mit `jsonlite::fromJSON()`: Aus der Liste rechts wird direkt eine Tabelle mit drei Zeilen.

Beispiel

```
[
  {
    "id": 1,
    "name": "van Pelt, Lucy",
    "geschlecht": "weiblich",
    "alter": 32
  },
  {
    "id": 2,
    "name": "Peppermint, Patty",
    "geschlecht": "weiblich",
    "alter": null
  },
  {
    "id": 3,
    "name": "Brown, Charlie",
    "geschlecht": "männlich",
    "alter": 27
  }
]
```

XML oder JSON?

	XML	JSON
Entstanden	1998, aus der Dokumentenwelt (SGML)	2001, aus der Webprogrammierung (JavaScript)
Umfang	Manchmal unnötig ausführlich, viel Markup	Kompakt
Schema und Validierung	Ausgereift (XSD), verpflichtend in der Verwaltung	Vorhanden (JSON Schema), weniger verbreitet
Typische Nutzung	Verwaltungsstandards (XÖV), E-Rechnung, Dokumentformate, ältere Schnittstellen	Web-APIs, Open-Data-Portale, Konfigurationsdateien, moderne Schnittstellen
Werkzeuge	Jede Programmiersprache; spezialisierte Validatoren	Jede Programmiersprache; direkt im Browser lesbar
Für Euch relevant	Was zwischen Fachverfahren und Portalen fließt	Was Ihr von Open-Data-Portalen und Web-APIs bekommt

✎ **Kein Entweder-oder**: Dieselben Daten lassen sich verlustfrei in beide Richtungen übersetzen. FIT-Connect, die Übermittlungsinfrastruktur des IT-Planungsrats, beschreibt jede Einreichung mit Metadaten nach JSON Schema und transportiert die Fachdaten wahlweise als JSON oder XML, je nachdem, was das Fachschema (FIM oder XÖV) vorgibt.

XÖV: die Datenstandards der deutschen Verwaltung

Das Problem

- Bund, 16 Länder, rund 11.000 Kommunen, hunderte Fachverfahren von dutzenden Herstellern. Wenn jede Stelle mit jeder eine eigene Schnittstelle baut, entsteht **„Schnittstellen-Spaghetti“** mit unbezahlbaren Wartungskosten.

Die Antwort

- **XÖV** (XML in der öffentlichen Verwaltung) ist ein **Standardisierungsrahmen**: gemeinsame Methoden, Bausteine und Entwurfsregeln, nach denen Fachstandards gebaut werden.
- Koordiniert von der KoSIT (Koordinierungsstelle für IT-Standards, Bremen) im Auftrag des IT-Planungsrats.
- Standards werden **XÖV-zertifiziert** und im XRepository veröffentlicht, samt Schemata und Codelisten.
- Gemeinsame Bausteine (Name, Anschrift, Datum, Behördenkennung) werden **wiederverwendet**.

Beispiele

Standard	Wofür
XMeld	Meldewesen: Datenaustausch zwischen Meldebehörden
XPersonenstand	Geburten, Eheschließungen, Sterbefälle
XBau	Bauanträge und Baugenehmigungen
XRechnung	Elektronische Rechnung an öffentliche Auftraggeber
xdomea	Austausch von Akten und Vorgängen (E-Akte)
XFall	Antragsdaten von Portalen in Fachverfahren

Beispiel XFall: ein Antrag auf dem Weg ins Fachverfahren

Hertie School

Worum es geht

- Das Land betreibt **ein Antragsportal** für alle Kommunen. Die Kommunen bearbeiten die Anträge in **hunderten verschiedenen Fachverfahren**.
- XFall legt fest, wie die Antragsdaten vom Portal ins Fachverfahren kommen: **ein Format, egal welcher Hersteller**.
- Entwickelt von fünf Ländern und der Stadt Münster, u.a. für Gewerbeanmeldung, Kfz-Ummeldung, Elterngeld, Führungszeugnis.

✎ Rechts: eine Gewerbeanmeldung, auf das Wesentliche gekürzt. Drei Blöcke, die in jedem XFall-Antrag vorkommen.

Die Fachnachricht (XFall-Daten 4.0.0)

```
<xfd:gewerbe.gewerbeanmeldung.05001002 xFallDatenVersion="4.0.0">

  <!-- 1. Wer schickt das, wann? -->
  <xfd:transport>
    <xfd:erstelltUm>2026-10-27T09:14:00</xfd:erstelltUm>
    <xfd:erstelltVon>Serviceportal Niedersachsen</xfd:erstelltVon>
  </xfd:transport>

  <!-- 2. Wer stellt den Antrag? (AS = Antragstellerin) -->
  <xfd:beteiligter>
    <xfd:rolle><code>AS</code></xfd:rolle>
    <xfd:vorname>Erika</xfd:vorname>
    <xfd:familienname>Mustermann</xfd:familienname>
    <xfd:anschrift> … </xfd:anschrift>
  </xfd:beteiligter>

  <!-- 3. Was wird beantragt? -->
  <xfd:ags><code>03159016</code></xfd:ags>            <!-- Göttingen -->
  <xfd:taetigkeit>Einzelhandel mit Fahrrädern</xfd:taetigkeit>
  <xfd:wzschluessel>47.64.1</xfd:wzschluessel>
  <xfd:datum>2026-11-01</xfd:datum>
  <xfd:anmeldegrund><code>NG</code></xfd:anmeldegrund> <!-- Neugründung

</xfd:gewerbe.gewerbeanmeldung.05001002>
```

Beispiel XFall: ein Antrag auf dem Weg ins Fachverfahren

Drei Dinge zum Mitnehmen

1. **Umschlag und Inhalt sind getrennt.** Der Container (links die Fachnachricht, rechts der Umschlag) trägt Absender, Empfänger, Anlagen und Frist. Er ist für alle Verfahren gleich; nur der Inhalt wechselt.
2. **Bausteine werden wiederverwendet.** Person, Anschrift, Kommunikation sind einmal definiert und tauchen in jedem Antragstyp gleich auf.
3. **Codes statt Freitext.** `AS`, `NG`, `03159016`: Jeder Code verweist auf eine Codeliste mit Version. Das Fachverfahren muss nichts interpretieren.

✎ Das Fachverfahren prüft die Nachricht gegen das Schema. Fehlt ein Pflichtfeld oder stimmt ein Code nicht, wird der Antrag gar nicht erst angenommen.

Der Umschlag (XFall-Container 3.0.0)

```
<xfc:application.transport.0300001>
  <xfc:header>
    <xfc:messageID>urn:uuid:3f6c2a1e- … </xfc:messageID>
  </xfc:header>

  <!— Absenderin: das Bürgerkonto —>
  <xfc:partner>
    <xfc:partnerID>buergerkonto-ni-00012345</xfc:partnerID>
    <xfc:authentNiveau><code>4</code></xfc:authentNiveau>
  </xfc:partner>

  <!— Empfänger: das Gewerbeamt —>
  <xfc:partner>
    <xfc:partnerID>03159016-gewerbeamt</xfc:partnerID>
    <xfc:orgName1>Stadt Göttingen</xfc:orgName1>
  </xfc:partner>

  <!— Anlagen und Frist —>
  <xfc:document><xfc:name>Ausweiskopie</xfc:name></xfc:document>
  <xfc:deadline><xfc:endDate>2026-11-27</xfc:endDate></xfc:deadline>

  <!— Der Inhalt: die Fachnachricht von eben —>
  <xfc:implementation>
    <xfd:gewerbe.gewerbeanmeldung.05001002>  …  </xfd: … >
  </xfc:implementation>
</xfc:application.transport.0300001>
```

`Quellen` XFall-Daten 4.0.0, XFall-Container 3.0.0; Beispieldateien in `materials/`.

Übung: Eine XFall-Nachricht auseinandernehmen

Öffnet diese Datei im Browser (eine echte, schema-valide Gewerbeanmeldung):

raw.githack.com/hertie-ai-certificate/modul-2-2026/main/d1s3-datengewinnung/materials/beispiel-gewerbeanmeldung_xfall-daten-4.0.0.xml

1. **Wer** meldet **was** an, und ab **wann**?
2. Findet drei Stellen mit `<code>…</code>`. Was bedeuten die Codes, und woher kommt die Liste?
3. Welcher Block wäre bei einer **Kfz-Ummeldung** genau gleich?
4. Was steht **nicht** drin, was ein Sachbearbeiter zum Bescheid noch bräuchte?

Jeder für sich, 5 Minuten. Die Kommentare in der Datei helfen.

Checkliste: Daten bereitstellen

Bevor Ihr einen Datensatz weitergebt

1. **Rechteck**: Eine Tabelle, eine Kopfzeile, keine Formatierung als Information.
2. **Format**: CSV (UTF-8, Trennzeichen angeben) für Tabellen; JSON oder XML für verschachtelte Daten. Excel höchstens zusätzlich.
3. **Datenwörterbuch**: Jede Spalte erklärt, Einheiten, Codes, fehlende Werte.
4. **Steckbrief**: Zweck, Erhebung, Zeitraum, Stand, bekannte Lücken, Ansprechperson.
5. **Kennungen**: Amtliche Schlüssel statt Namen, wo es sie gibt.
6. **Lizenz**: Explizit nennen, auch intern.
7. **Ort**: Eine stabile Adresse, keine E-Mail-Anlage.
8. **Datenschutz**: Personenbezug geprüft, ggf. aggregiert oder anonymisiert, Freigabe dokumentiert.

Und die Gegenprobe

- Könnte eine Kollegin aus einem anderen Haus den Datensatz **ohne Rückfrage** nutzen?
- Könnte ein Programm ihn **ohne menschliches Zutun** einlesen?
- Wüsste jemand in fünf Jahren noch, **was die Zahlen bedeuten**?

✎ Mehr zum Umgang mit Open Data, Veröffentlichung und Nutzung an Tag 3!

